INS 204 GROUP 7

CHARLES DANIEL CHINEDUM 24/1960
ONWUATUEGWU CHIMAOBI 24/1256
AYOOLA LUCAS 24/0104
EMONEFE TREASURE 24/0005
EZEIBE OSINACHI HILARY 24/0722
BANTEFA GBOLAHAN 24/0090
IGBAH DAVID 24/0281

INS PRESENTATION GROUP 7

FRIDAY 3RD APPRIL 2026

INTRODUCTION

WHAT IS OBJECT ORIENTED ANALYSIS & DESIGN

Object-Oriented Analysis & Design (OOAD) is a foundational software development approach that has significantly transformed how software systems are conceptualized, structured, and implemented.

It models a system as a collection of interacting objects that reflect real-world entities, making complex systems easier to understand, design and maintain. OOAD integrates two closely related phases:

- Object-Oriented Analysis
- Object-Oriented Design

Object Oriented Analysis

This focuses on understanding system requirements by identifying relevant object, their responsibilites, and how they interact within a given problem domain.

Think of it as understanding the requirements and the problem domain using objects as the main building blocks.

Object Oriented Design

Object-Oriented Design is the process of taking the analysis models (from OOA) and defining how the system will be built using object-oriented principles.

In other words, OOD is about how the system will work, not just what it should do

Object Oriented Analysis & Design
— (OOAD) —

OOAD is a structured approach to software development that uses object oriented principles to understand a problem domain (analysis) and design a solution (design).

ANALYSIS
Understand the Problem Domain

1. **Identify Requirements**
   Gather and understand user needs and system goals.
2. **Model the Domain**
   Identify relevant objects, attributes, relationships and constraints.
3. **Create Analysis Models**
   - Use Case Diagrams
   - Class Diagrams (Conceptual)
   - Sequence Diagrams (High-Level)
   - Activity Diagrams
4. **Validate the Model**
   Ensure the model accurately represents the real-world requirements.

DESIGN
Define the Solution

1. **Design Classes**
   Define classes, attributes, operations and relationships in detail.
2. **Design Interactions**
   Specify how objects interact using sequence, collaboration and state diagrams.
3. **Design Architecture**
   Define the system's structure, layers, components and data persistence.
4. **Refine & Optimize**
   Improve the design for performance, reusability, security and maintainability.

OUTPUT

A complete design specification ready for implementation

- Class Diagrams
- Sequence Diagrams
- Database Design
- Interface Design
- Other Design Artifacts

BUILT ON OO PRINCIPLES

Abstraction | Encapsulation | Inheritance | Polymorphism

BENEFITS

Better Problem Understanding | Reusable & Maintainable Design | Reduced Development Time & Cost | Higher Quality Software

From Understanding the Problem to Designing the Right Solution

Important Aspects Of OOAD

Some important aspects of Object Oriented Analysis & Design are:

a. Object oriented Programming: OOAD is based on object-oriented programming concepts in which real-world entities are represented as software objects.

b. Design Patterns: Design patterns are reusable solutions to commonly occuring design problems. In OOAD, design patterns guide developers in creating software systems that are efficient, flexible, and easy to maintain by following peroven best practices.

c. Unified Modeling Language (UML): UML diagrams are used in OOAD to visually represent the structure and behavior of a software system.

d. Use Cases: These describes how users interact with a system to achieve specific goals. OOAD uses use cases to capture user intent and ensure the software design meets user and business needs.

Object Oriented Analysis And Design SDLC

The Object-Oriented Analysis and Design system development life cycle follows a structured, iterative, and incremental approach to software development. Although the phases are presenteded in a logic sequence, feedback from later stages may require revisiting and improving earlier phases.

a. Requirement Analysis: The requirement analysis phase focuses on understanding user needs, system goals, and operational constraints.

b. Object-Oriented Analysis (OOA): This phase focuses on understanding and modeling the problem domain using object-oriented concepts.

c. Object-Oriented Design (OOD): This phase transforms object-oriented analysis odels into detailed design specifications that define the structural solution of the system.

Object Oriented Analysis And Design SDLC

d. Prototyping: This involves developing an early working version of the software or specific system components.

e. Implementation: During implementation, the system design is translated into executable code using an Object-Oriented Programming Language such as Java or C++

f. Incremental Testing: Testing in OOAD is performed incrementally and continuously throughout development. It includes unit testing of individual classes, integration testing of interacting objects, and system testing of the complete application

g. Deployment and Maintenance: This involves releasing the system for operational use and making it available to end users and ensuring continued system effectiveness after initial release by fixing defects, adapting to changing requirements, improving performance and enhancing functionality.

RL APPLICATION

BENEFITS & CHALLENGES OF OBJECT ORIENTED ANALYSIS & DESIGN

PART TWO SUMMARY

in part one, we successfully introduced the topic, Object-Oriented Analysis & Design

In this second part, we will examine the real-world applications of OOAD, highlighting both its practical benefits and the challenges encountered when implementing object-oriented principles in actual software solutions.

Benefits Of OOAD

1. Modularity and Maintainability: OOAD encourages the decomposition of complex systems into smaller independent systems into smaller, independent objects that encsapsulate both data and behavior.

2. Reusability & Code Quality: By emphasizing self-contained classes and objects, OOAD promotes code reuse across different parts of an application or across multiple projects.

3. High-Level Abstraction: OOAD provides an abstract view of the system, allowing developers to focus on essential system features without being overwhelmed by implementation details.

4. Scalability & Adaptability: Object-oriented systems can easily scale and adapt to new requirements. New functionality can be introduced by extending existing classes or adding new objects without major changes to the existing system

Benefits Of OOAD

5. Real World Modeling: OOAD closely maps software components to real-world entities such as users, products, or bank accounts.

6. Improved Communications and Collaboration: Standard modeling tools such as UML provide a common visual language that enhances communication among team members and supports collaborative development across different system modules.

7. Lower Cost Of Development: Although OOAD may require more effort during the analysis and design phases, this upfront investment typically leads to faster development, fewer errors, and reduced maintenance costs in the long run

Challenges Of OOAD

1. System complexity: OOAD systems are built around multiple objects and their interactions, which can make the overall system structure complex.

2. Complexity in Planning and Design: The analysis and design phases of OOAD require careful identification of objects, classes and relationships.

3. Steep Learning Curve and Skill Requirements: OOAD requires strong knowledge of object-oriented concepts, design principles, and design patterns.

4. Performance Overhead: The use of abstraction, encapsulation, and object instantiation can introduce performance overhead.

Challenges Of OOAD

5. Time Consuming Development Process: OOAD emphasizes thorough upfront analysis, design and documentation. While beneficial in the long term, this can result in longer development timelines, especially for projects with tight deadlines.

6. Higher Development Cost: The need for extensive planning, documentation, skilled personnel, and longer development cycles can make OOAD more expensive compared to simpler development methodologies.

7. Overhead of Reusability: Designing systems with reusability in mind often requires extra effort during the design phase.

8. Integration With Legacy Systems: Integrating object-oriented systems with existing legacy or procedural systems can be difficult.

Real World Applications

Object-Oriented Analysis and Design (OOAD) is a powerful widely adopted methodology used across many industries to develop robust, scalable, and maintainable software systems.
Some key real-world applications of OOAD are outlined below:

a. Banking and Financial Services: OOAD is extensively used in banking software to model complex financial structures, customer accounts, transactions, loans, and compliance processes.

b. Electronic Health Record & Healthcare Systems: In healthcare applications, OOAD is used to model patient data, medical histories, appointments, billing, and clinical workflows.

c. E-Commerce and Online Shopping Platforms: OOAD plays a critical role in the design of e-commerce systems by modeling products, users, shopping carts, orders, payments and delivery services.

Real World Applications

d. Flight Control and Aviation Systems: In aviation, OOAD is essential for designing flight control systems that require high reliability and precision. It helps model interactions among sensors, naviagtion systems, control surfaces, and montoring components.

e. Gaming Industry: Game development heavily relies on OOAD to manage characters, game environments, physics engines, and player interactions.

f. Educational Software and Learning Management Systems: Educational platforms use OOAD to build interactive and adaptive learning environments. Objects represent students, courses, assessments, and instructors.

PRESENTATION ASSIGNMENT

PART THREE

SUMMARY OF CHAPTER 7 – 10

Omo mehhn

INS 204 GROUP 10

Chapter 7: Coding, Testing and Implementation

This chapter teaches in detail, about Systems Implementation.
A process which includes hardware acquisition, programming and software acquisition or development, user preparation, hiring and training of personnel, site and data preparation, installation, testing and start-up (changeover procedure) and user acceptance.

It also covers areas such as:

1. Acquisition Standard: This refers to the rules and guidelines an organization follows when buying or obtaining new hardware, software, or IT services for a system.

2. Hardware and Software Acquisition: This is the process of selecting, purchasing, and obtaining the physical devices and software programs needed for an information system.

Chapter 7: Coding, Testing and Implementation

3. Coding: This is the process of writing instructions in a programming language to develop software that performs specific tasks

4. Software Testing: is the process of examining and evaluating a software system to determine whether it functions correctly and meets specified requirements. Types of software testing include unit testing, integration testing, system testing, acceptance testing, performance testing, security testing, and regression testing.

5. Documentation: This is the process of creating written records and supporting materials that describe how a system is designed, developed, used, maintained, and managed.

6. System deployment: This is the process of installing, configuring, and making a completed system available for users to operate in a real working environment.

Chapter 8: Systems Evaluation & Maintenance

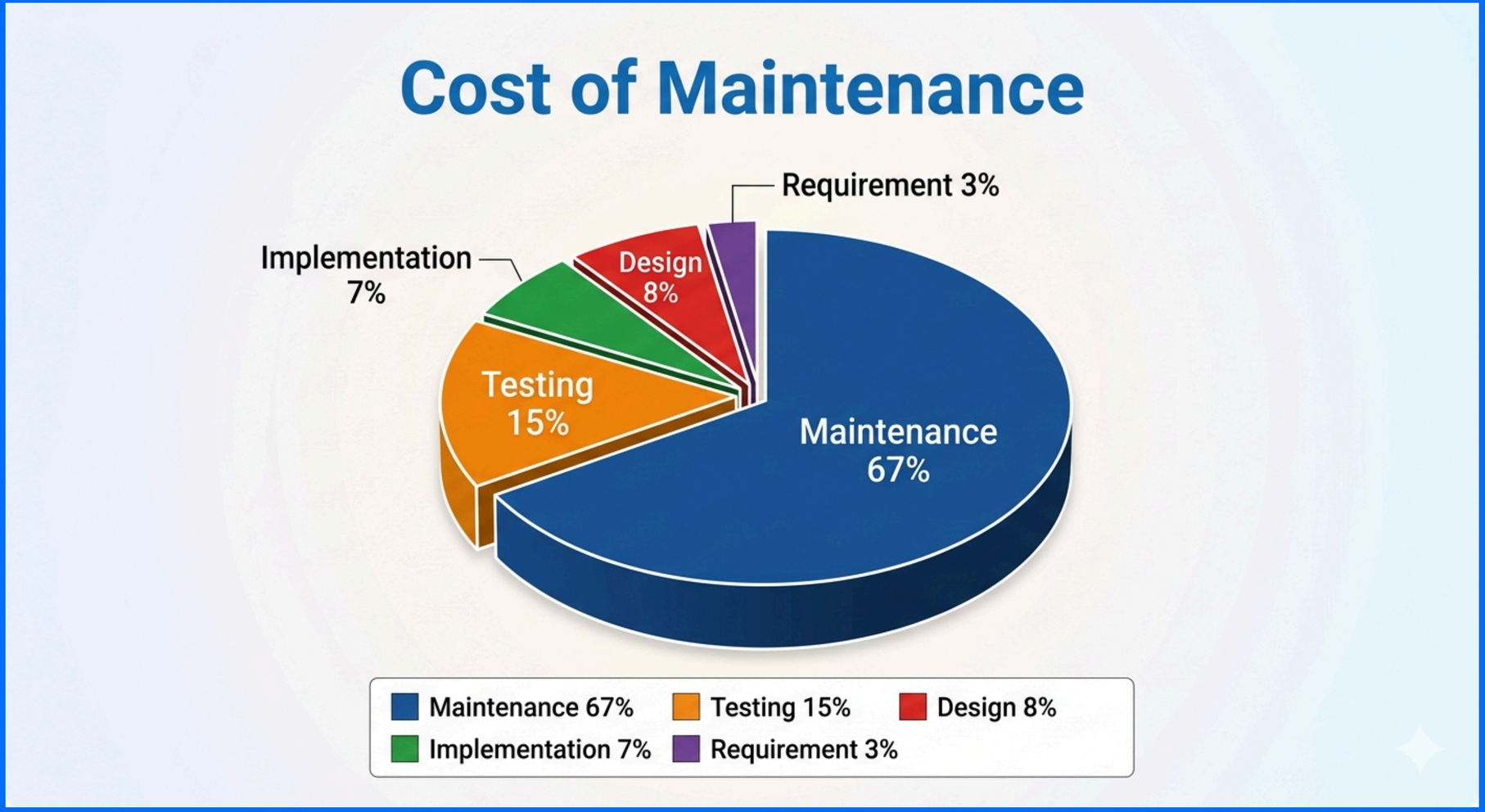
Systems evaluation is the process of assessing a completed information system after it has been implemented to determine whether it is meeting its intended objectives, user requirements, and organizational needs. It involves examining the system's performance, reliability, security, efficiency, ease of use, and overall effectiveness in solving the problem it was designed for.

During systems evaluation, feedback is collected from users, errors and weaknesses are identified, and the costs and benefits of the system are reviewed to see whether the system is delivering the expected value.

System maintenance is the process of updating, correcting, improving, and supporting a system after it has been implemented to ensure that it continues to function properly and meet user needs.

Maintenance is often the most expensive stage of the system life cycle and can account for about 60% of the total cost of a system because continuous support and updates are usually required over a long period of time.

Chapter 8: Systems Evaluation & Maintenance



Chapter 9: Introduction to OOAD and UMLs

This chapter touches the surface of the object-oriented programming approach.

The object-oriented paradigm in OOAD organizes a system around objects instead of functions or procedures, where each object encapsulates both data and behaviors representing real-world entities. Its goal is to model software in a way that reflects real-world structures and interactions. This approach enhances clarity, reusability, and maintainability of the system.

Chapter 9: Introduction to OOAD and UMLs

Some key concepts of the Object-Oriented Approach:

1. Objects: These are basic runtime entities in an object-oriented system. They represent real-world entities such as bank accounts,, employee, vehicle or student.

2. Classes: A class is a blueprint, template, or logical structure used to create objects. It defines common attributes and methods that all objects derived will possess.

3. Encapsulation: This is the mechanism of bundling data and the methods that operate on that data into a single unit, usually a class.

4. Inheritance: This is a mechanism that allows a new class, known as a child, to acquire the attributes and behaviours of an existing class called a parent or superclass.

5. Others include Polymorphism, Modularity, Constructors and Destructors

Chapter 10: Information System Development

This covers everything we already covered in Part One and Two

References

1. Systems Analysis And Design Textbook [Principles, Practices and Applications]
2. ChatGPT